Programming

Mr Butler | Year 10 ICT



What is programming?

- Programming is everything involved in making, controlling and communicating with digital devices.

- Programming is often called ‘coding’, as it involves writing a language that computers can understand. This allows us to develop devices, apps and technology in our own way.

- A **programmer** is a person who writes code for a device and is also involved in testing the code to ensure it is fully functional.



Q1

Who is considered to be the first ever computer programmer?



Q2

How many programming languages are there to use?

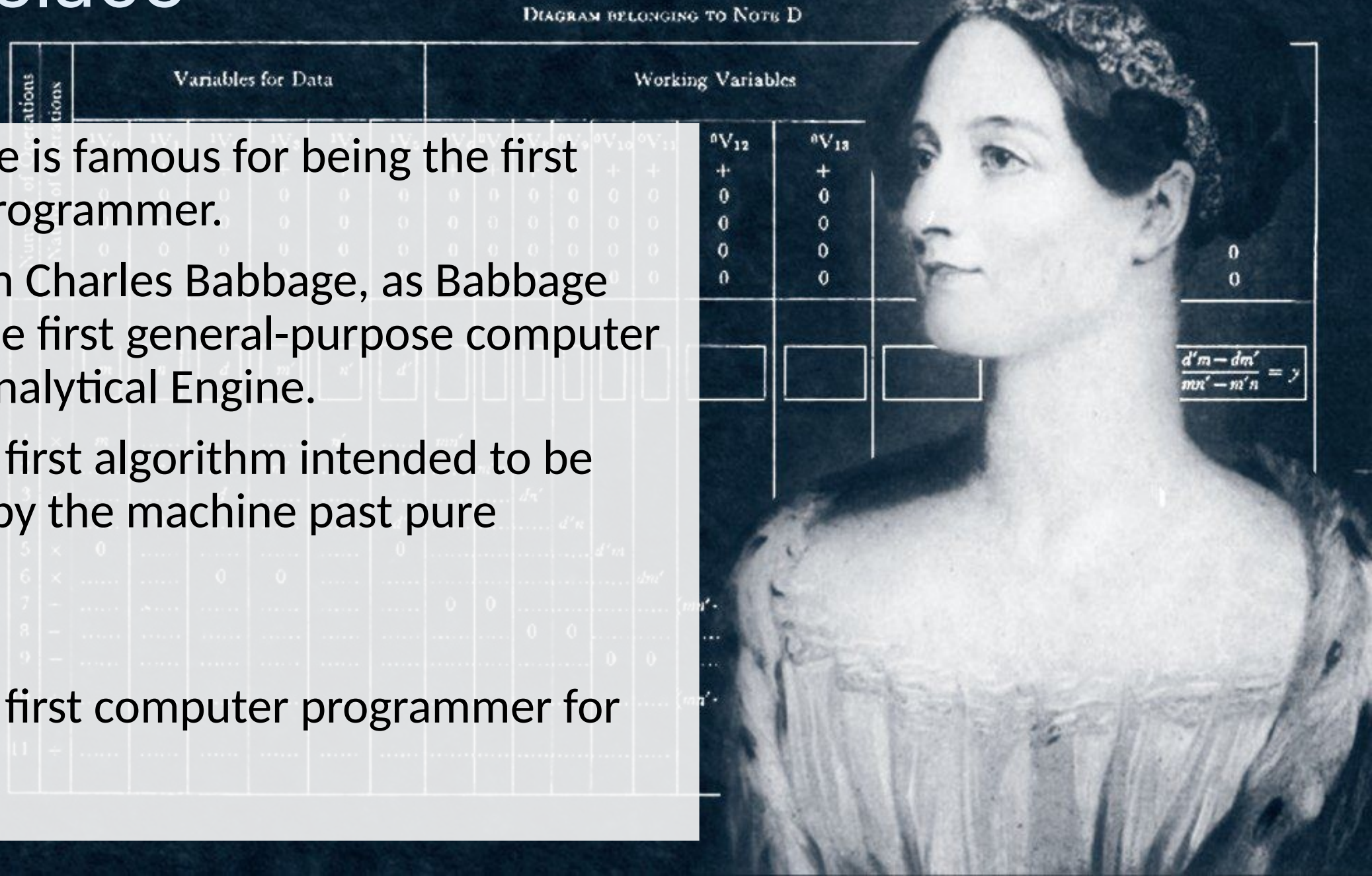


Q3

What jobs can you get from knowing programming ?

Ada Lovelace

- Ada Lovelace is famous for being the first computer programmer.
- Worked with Charles Babbage, as Babbage proposed the first general-purpose computer called the Analytical Engine.
- Devised the first algorithm intended to be carried out by the machine past pure calculation.
- Cited as the first computer programmer for this reason



The different programming languages

- There are roughly 700 programming languages used today.
- These all have different uses and use cases
- Some are very good for:
  - App development
  - Web development
  - Phone/tablet development
  - Operating system development
  - Software Development
- Programming languages appear for different jobs as each have their own features, benefits and characteristics suitable for different things.

The Jobs Of Programming

- We all want a well paid job
- Jobs in IT are developing and have developed considerably every year.
- IT jobs are going to make you a LOT of money.



Are there a lot of programming jobs?

Coding jobs are in demand, and the Bureau of Labor Statistics (BLS) projects that computer and information technology employment will grow 13% between 2020 and 2030, which is faster than the average projected growth rate across all occupations (8%).

21 Dec 2021



IT Manager

#13 in **Best Paying Jobs**

Our increasingly digital workplace demands more IT managers, who coordinate computer-related activities for an organization. Duties include analyzing and recommending computer needs, installing and maintaining computer hardware and software, securing an office's network and electronic documents and searching for new technologies and upgrade opportunities. **READ MORE »**

PROJECTED JOBS
52,700

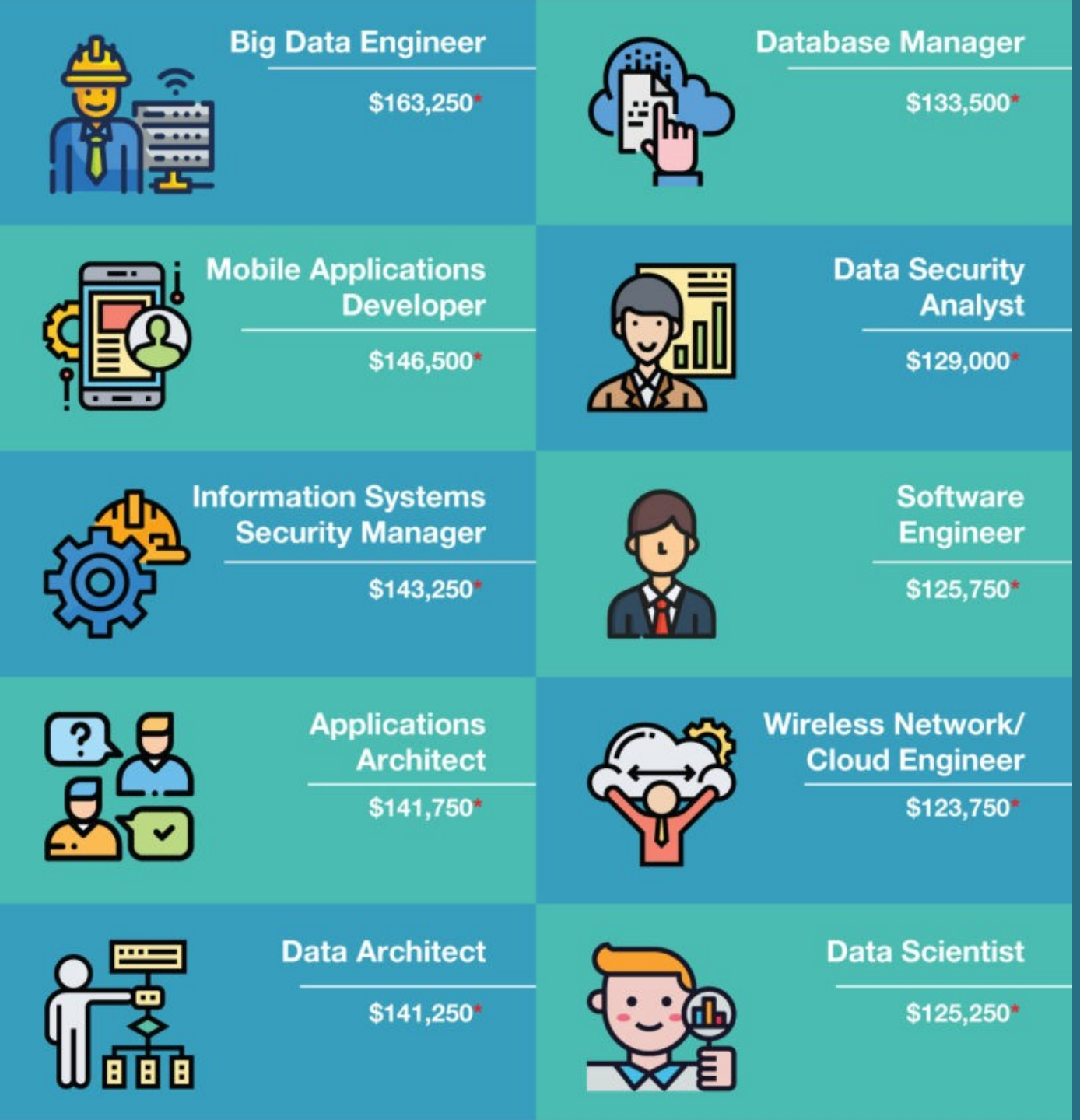
MEDIAN SALARY
$151,150

EDUCATION NEEDED
Bachelor's

See this number?

Whilst #13 in highest paying jobs, the projected number of jobs far exceeds anything above it. In fact, the other projected jobs above this are reducing by hundreds!

Source: https://money.usnews.com/careers/best-jobs/rankings/best-paying-jobs



JOBS IN IT/PROGRAMMING

There are so many jobs in IT, including:

- Software engineer
- Network architects
- IT Project managers
- Database Administrators
- Web Developers
- System Analyst
- Mobile App Developer
- IT Consultant
- Technical Sales Rep
- Support Analyst
- Cyber Security Specialist
- Social Media Manager

- Medicine/Dentistry – The analysis skills help!
- Army – The problem solving is perfect!
- Analyst – Literally analysing everything!
- Marketer – All of the above!

WHY LEARN PROGRAMMING?



- Not only does it allow you to give machines instructions, but it also teaches you **abstract thinking**, **pattern recognition** and **problem-solving**

- These are key skills you will use over the rest of your life.
  - Abstract thinking: ability to work with concepts rather than physical things
  - Pattern recognition: ability to see relationships and consequences from actions that haven't happened yet
  - Problem solving: ability to find solutions to everything and anything, and be able to think outside the box in many situations (also known as forward thinking)

The Many Uses of Programming

- Think of all the different computing devices you have used – phones, tablets, computers, gaming consoles, docking stations, credit card readers.

- All of these had to be designed, developed, tested and produced in the first place. The role of a computer programmer is to design, write and test the underlying code within these devices to make them work!

TASK

Pick a device you have at home. Find out who designed the device and programmed it. Create a small credit card sized business card for this person, company or group of people.

What programming do we already know?

- In ICT class, we have used:
  - Scratch – drag and drop event programming
  - BBC Microbits – drag and drop event programming
  - HTML/Web Development – The first introduction to a lot of computational thinking happened here
- In wider school, you would have used:
  - Methods in Science (SEQUENCING)
  - Made decisions in Art (SELECTION)
  - Used count-controlled loops and condition-controlled loops in PE (REPETITION)

- It is now time for you to put this altogether and learn a **programming language**!

Let's Learn a great programming language – C#

- C# is a high-level language, which means it is very close to 'human readable English'
- Other programming languages are low level languages, and these are close to looking like 1's and 0's (binary)
- A **compiler** is the computer program that converts our 'human code' into the binary code computers can understand – we don't usually write in binary.

How are we going to learn C#?

- As we know from KS3, **virus'** are small pieces of code written to damage the operation of a computer, steal data or corrupt data.

- C2K, who manage our computers, try to ensure they are as secure as possible and so limit what we can code on computers. At GCSE we can use a **virtual machine** which allows us to code without the risk of breaking school computers.

- This year, however, we are going to use an online tool to learn C#.

What can C# do?

- C# is an elegant object-orientated language that enables developers to build a variety of secure and robust applications that run on the **.NET Framework**.

- For example, the mobile game Pokemon Go and the Stack Overflow website were built with frameworks that can be run with C# (Unity and ASP.NET, respectively).

- You can use C# to create Windows applications, Web services, mobile applications, and much much more

Quick Question: C# applications run:

A: Only on Linux

B: on the .NET Framework

C: Using Java

D: None of these

Why use a strongly typed language like C#?

- While it means writing more code, using types has long term benefits like **built-in documentation** and increased **readability**.
- It also allows us to use an extensive **intellisense** – which basically is predictive text when you are writing code.
- Intellisense will also offer you help with **debugging** as it knows what can and can’t happen with a data type before you even try to run your code!

Mastering C# with Visual Studio



A Journey from Block Programming to Professional Code

Think Like a Programmer

From Scratch to C#

You have used **Scratch** to drag and drop blocks. Programming is just the path from an *idea* to a *solution*.

C# (pronounced 'C-Sharp') is a professional language used by companies like Microsoft, NASA, and game developers using Unity.

It’s the primary language used to build almost every game you see made in the Unity Engine, including Pokemon Go and Among Us.



What is Visual Studio?

To write C#, we use an **Integrated Development Environment (IDE)** called Visual Studio.

Allow a programmer to view, write, test and run code



Features Include



Line numbering helps programmers distinguish between lines in long programs (Very helpful in debugging)



Colour coding of keywords: Also different coloured underlines – red underline, for example, displays a syntax error



Clipboard (for copy + paste)

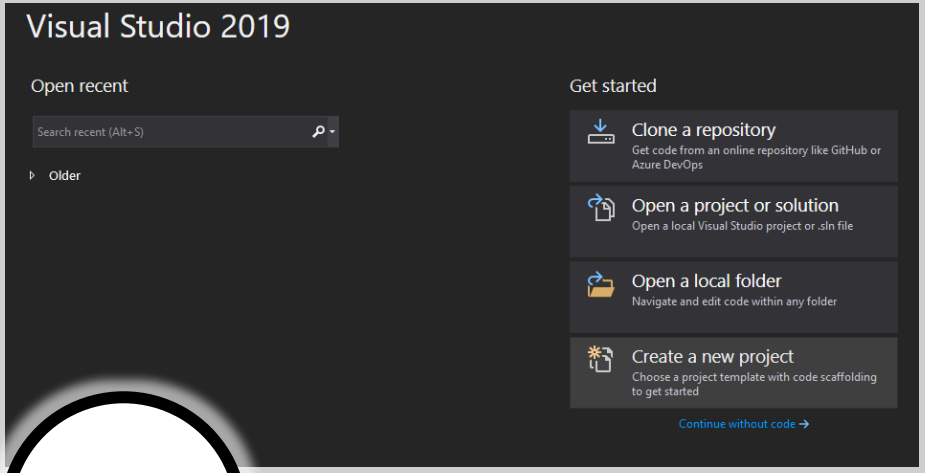


Collapsible code sections: Makes viewing large programs easier

Starting Your First Projec

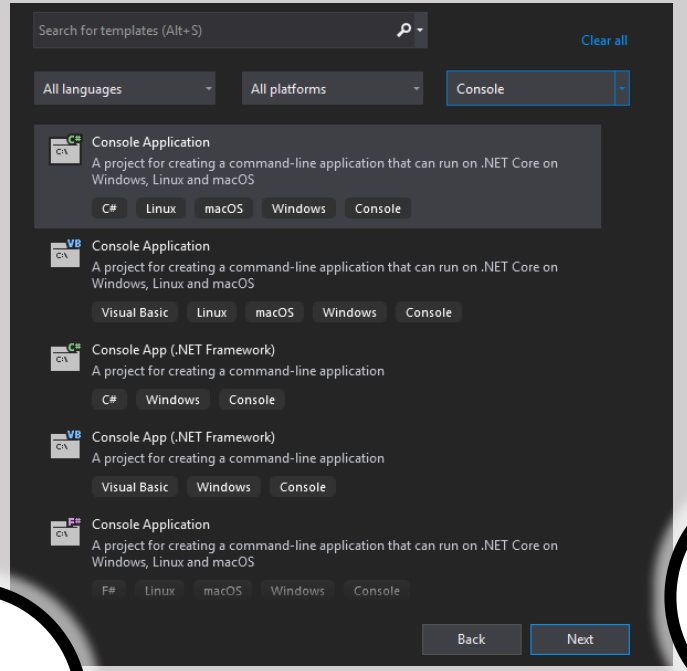
A **Console application** uses a text-only interface. We chose this type of application to focus on learning the fundamentals of C#.

- Choose a **Console Application** to create a simple, text-based program.



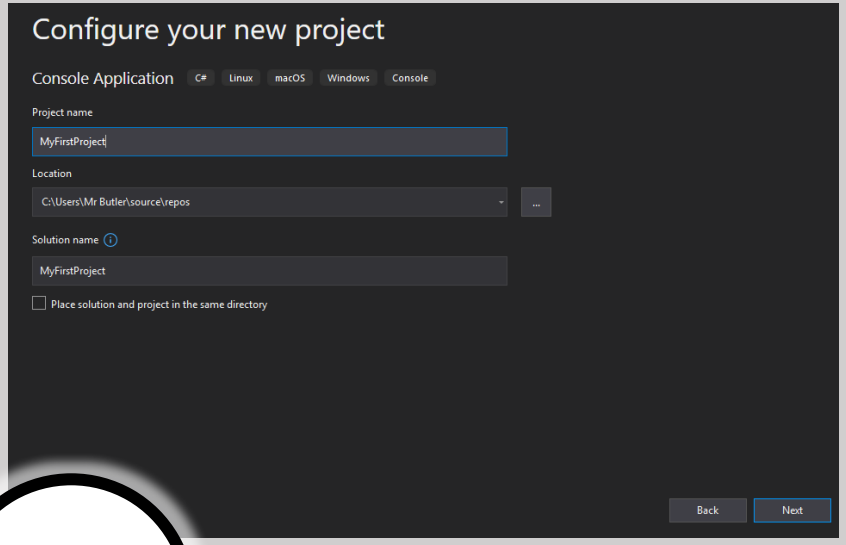
1

Select Create a new Project



2

Template: Select Console Application – ensure it is the C# version!



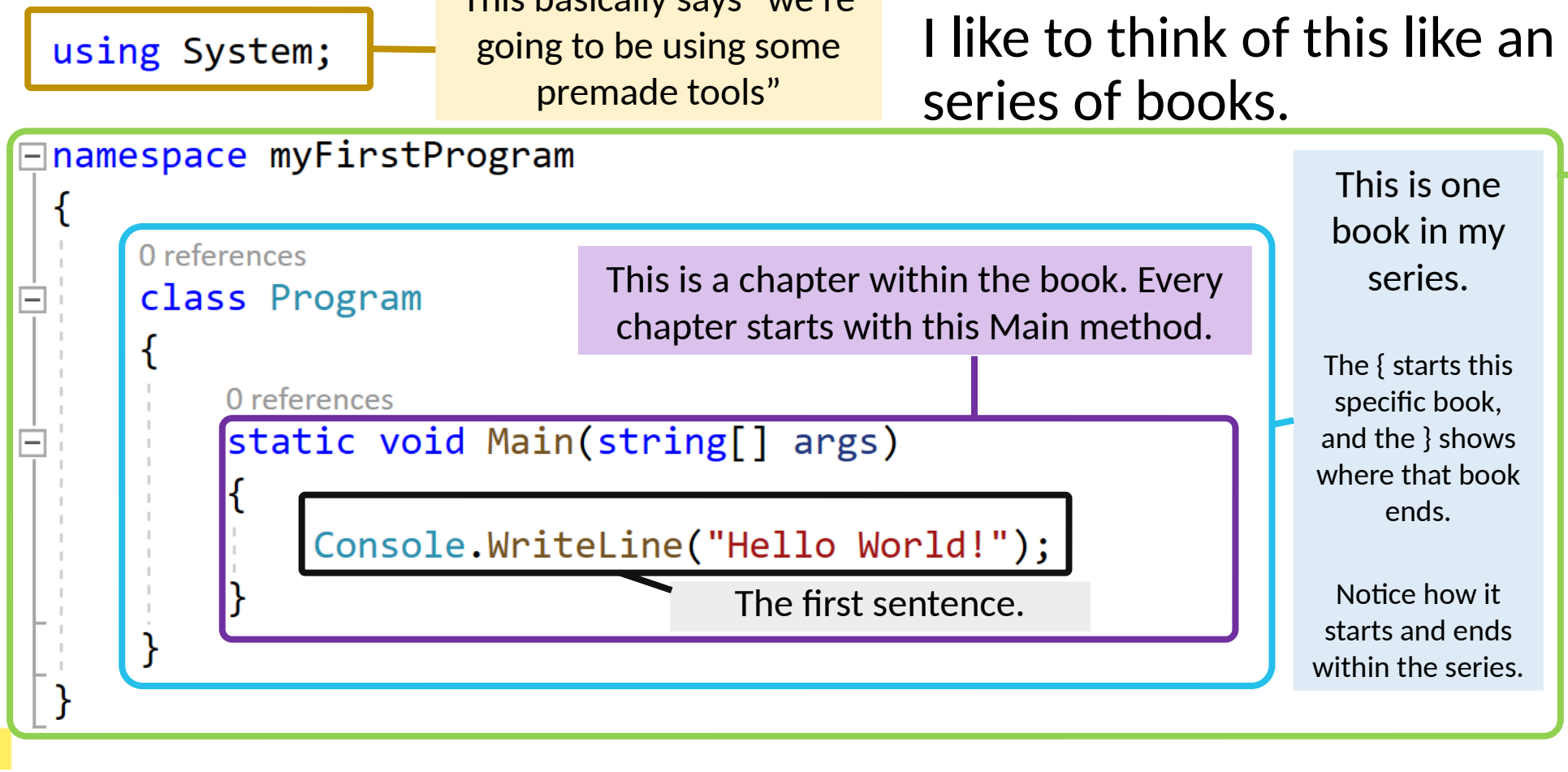
3

Enter a project name (no spaces) and click Create.

A Program Structure

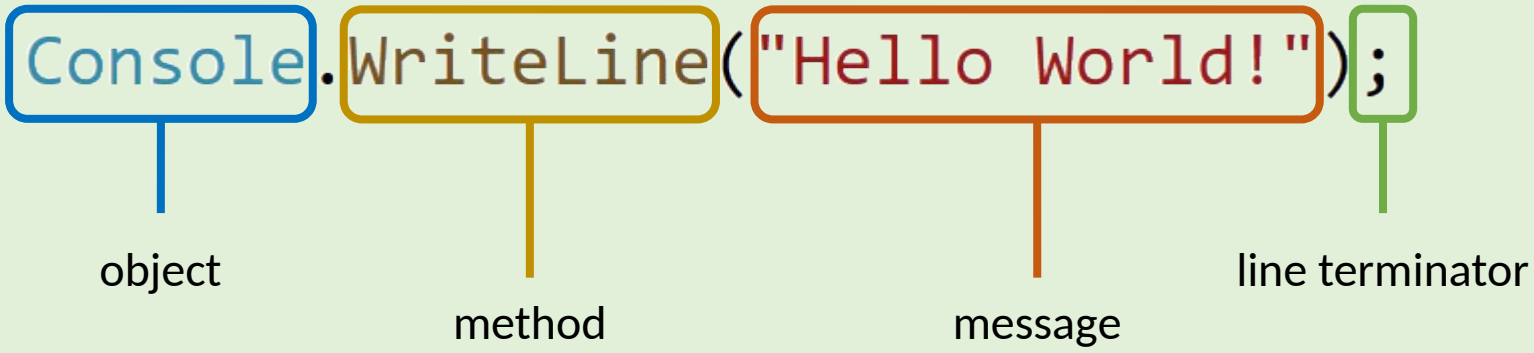
We're not going to go into too much detail here, but we just need to know the basics.

I like to think of this like an author of a series of books.



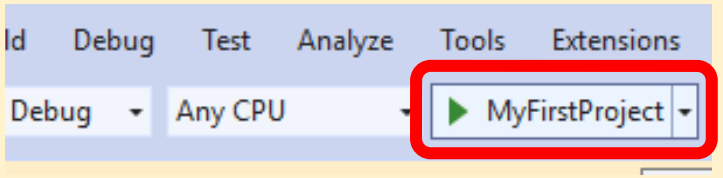
The Magic Command

To make text appear on the screen, we use this command:



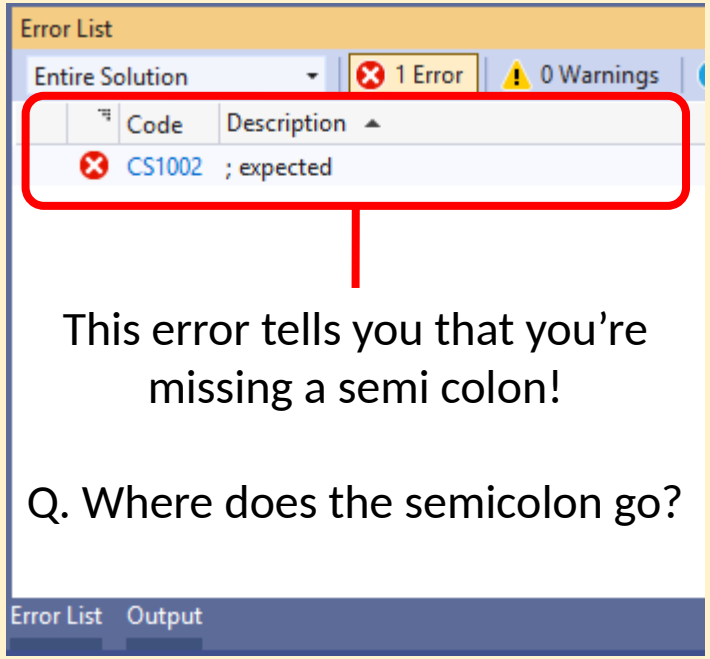
Remember
Every instruction must end with a semicolon (;). It is like a full stop at the end of a sentence in English.

Running Your Code



How to Start

To run your program, look for the **Green Play Button** at the top that says 'Start' or your project's name.



This error tells you that you're missing a semi colon!

Q. Where does the semicolon go?

If you see errors in the 'Error List' at the bottom, your program won't run. Treat errors as clues to a puzzle!

Challenge: Personalise it

Level 1

Display your **Full Name**.

📖 Example

```
Console.WriteLine(“I Like Coding”);
```

Level 2

Display your **Favourite Food**.

Level 3

Display your **Favourite Sport**.

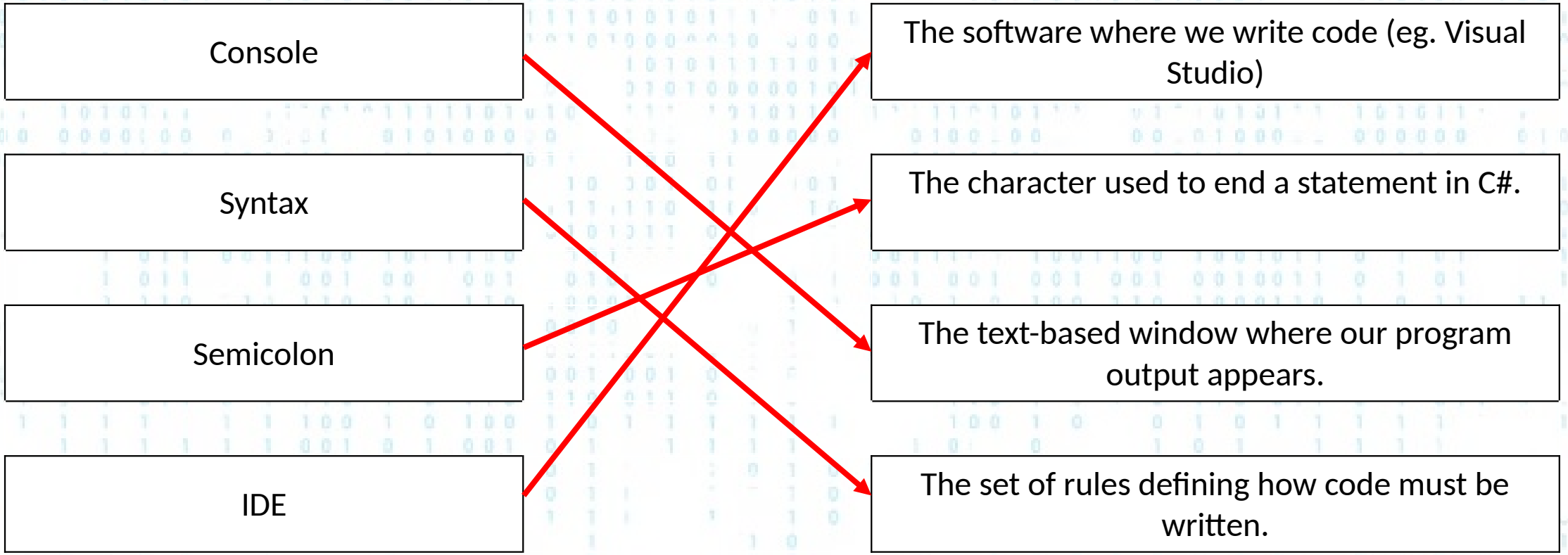
Adding a Comment

In programming, we often want to be able to add explanations to our code, often allowing other programmers see what a piece of code is doing.

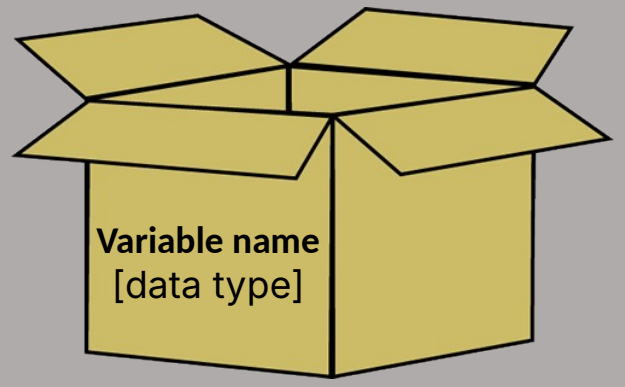
The computer needs w way to know the text you are writing is a comment, so we add // to the start to tell the computer *this is commentary*. For example:

```
//outputs a hello message to the user
Console.WriteLine("Hello World!");
```

Key Terms Review



The Box Analogy



Variables are like Boxes

Think of a variable as a storage box in the computer's memory. To use it, you need two things:

1. **A Name:** This is the variable name or what the box is labelled as.
2. **A Data Type:** This determines what can fit inside the box (e.g., an integer or text).

🚧 **Remember**
You must define the data type in C# before you can put anything in the box!

The Big Three Data Types

- In C#, we must specify the data type being stored. Here are the most common types:



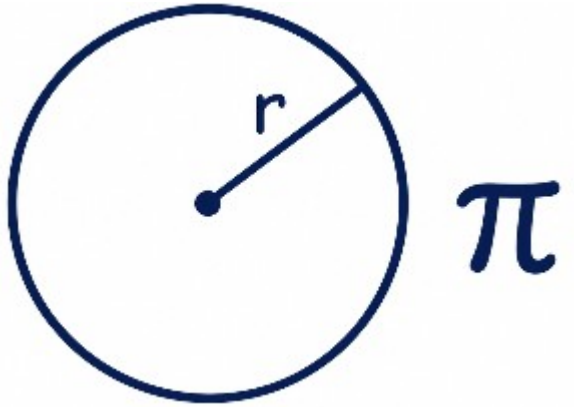
int

Short for Integer. Used for whole numbers like 10, -5, or 1,000,000.



string

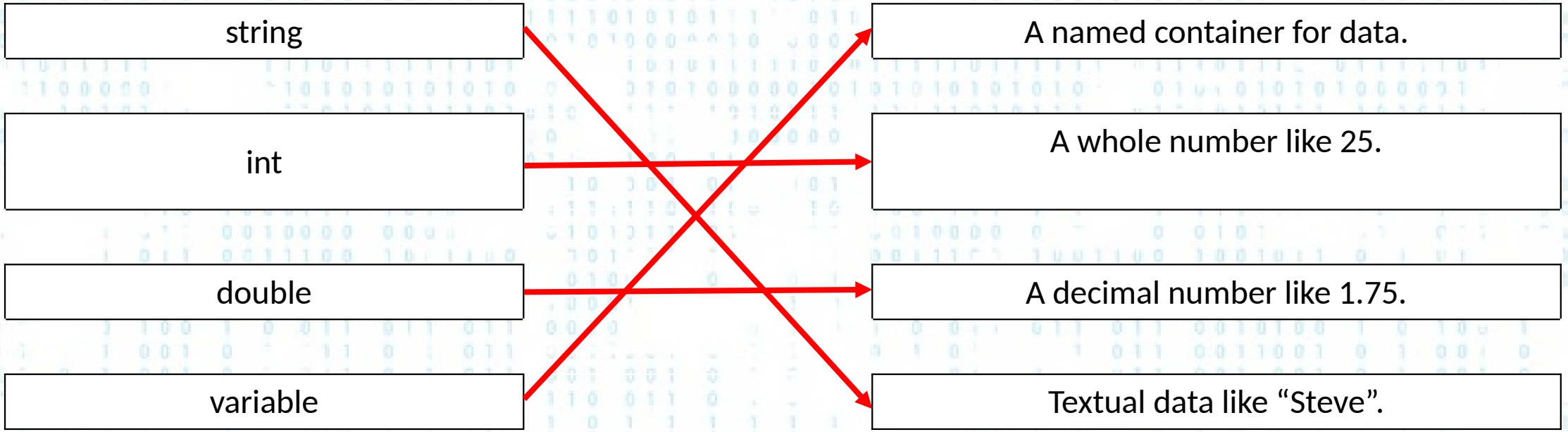
Used for text. Must be wrapped in double quotes (“ and ”), like “Hello World!”



double

Used for decimals. Perfect for precise values like 3.14 or 99.99

Match the Data!



Declaring & Assigning

The Syntax

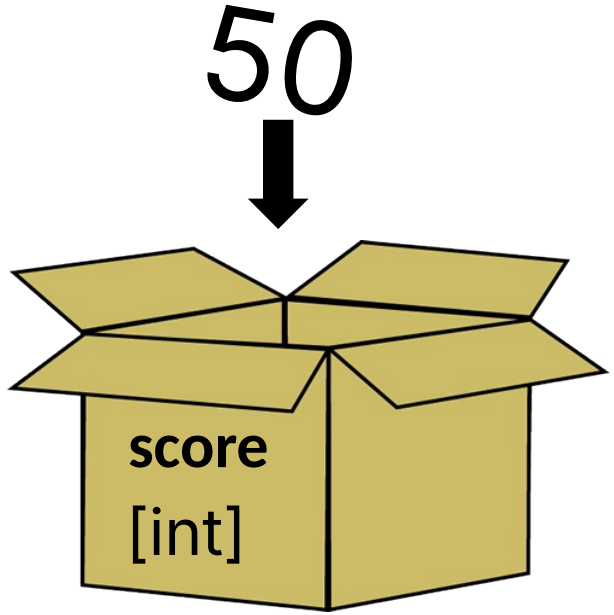
To create a variable, follow this pattern:

```
int score = 50;
```

data type

variable name

value



🚧 **Remember**

Don’t forget the semicolon (;) at the end of the statement. It tells the computer the instruction is finished.

Fix the code

Below is a screenshot of Tom, who is trying to declare three variables.

Correct the code by entering in a suitable data type for each variable

```
static void Main(string[] args)
{
    username = "Alice";
    price = 19.99;
    year = 2027;
}
```

🚧 **Remember**

Declaring variables always start with stating the data type first! i.e.

int age = 41;

Displaying Your Data

Console.WriteLine()

You can combine text and variables using the + symbol (this is called concatenation).

If the variable is declared as

```
string profession = "Coding Wizard";
```

Writing

```
Console.WriteLine("Hello " + profession);
```

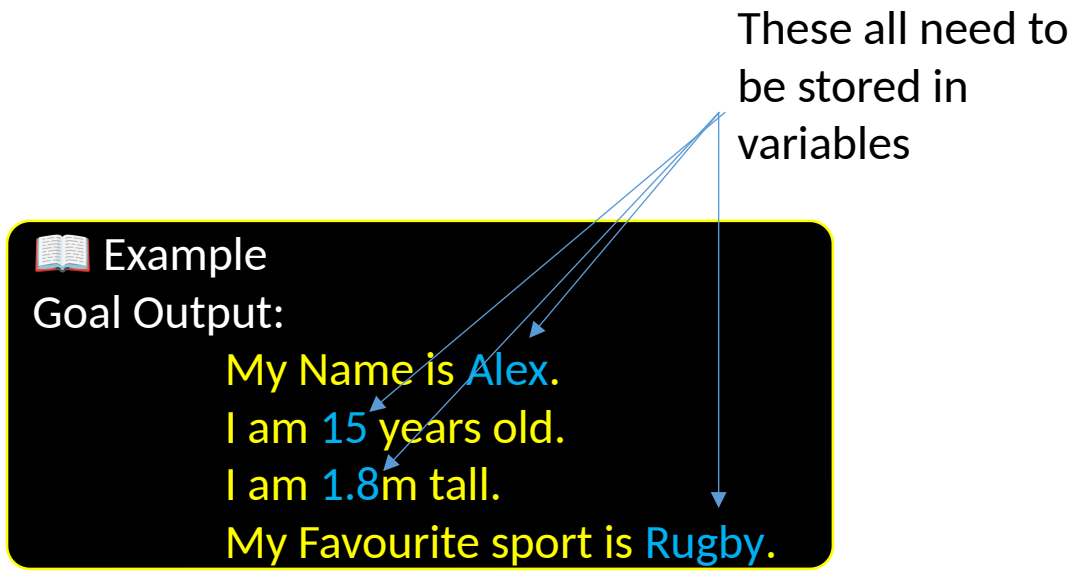
Will output

```
Hello Coding Wizard
```

Coding Challenge: Digital ID Card

The Task

Write a program storing profile information in variables, then print it.



📘 Extension

Goal Output:

My Name is Alex.
I will be 25 years old in 10 years time.
I am 1.8m tall.
My Favourite sport is Rugby.

Why Bother with Data Ty 

If we could just make every variable a 'string', why does C# force us to use 'int' and 'double'? What would happen if we tried to add two 'strings' together like "10" + "10"?

You might have said...

Computers process numbers (int/double) differently than text (string).

If we use string for numbers, we can't do maths properly (e.g. "10" + "10" would result in "1010", not 20).

Using specific types saves memory and prevents errors in complex calculations.

Activity: What's wrong here then?

TASK

There are 5 errors in this code snippet. Copy out this snippet and highlight/circle the errors.

Annotate how or what the correct solution should be!

```
1   using System;
2
3   namespace BugSquasher
4   {
5     class Program
6     {
7       static void Main(string[] args)
8       {
9         int number = 38498.3222;
10
11        dinosaur = "Barney";
12
13        double lock = 293.000;
14
15        bool is.yes = true;
16
17        string band = "The Low Anthem"
18
19      }
20    }
21  }
```

Solution

There are 5 errors in this code snippet. Copy out this snippet and highlight/circle the errors.

Annotate how or what the correct solution should be!

```
1   using System;
2
3   namespace BugSquasher
4   {
5     class Program
6     {
7       static void Main(string[] args)
8       {
9         int number = 38493.3222;
10
11        dinosaur = "Barney";
12
13        double lock = 293.000;
14
15        bool is.yes = true;
16
17        string band = "The Low Anthem"
18
19      }
20    }
21  }
22
```

An integer (int) must be a whole number

A variable must be declared with a specific type

A variable name cannot have a full stop/symbol

A double can only have 2 decimal places

A statement must end with a semi colon (;)

Mark yourself out of 5!

Making Programs Interactive in C#

Imagine a world where apps didn't listen to you.

No search bars, no login screens, and no game controls. Boring, right? To make software useful, we need to gather data from the user and react to it.

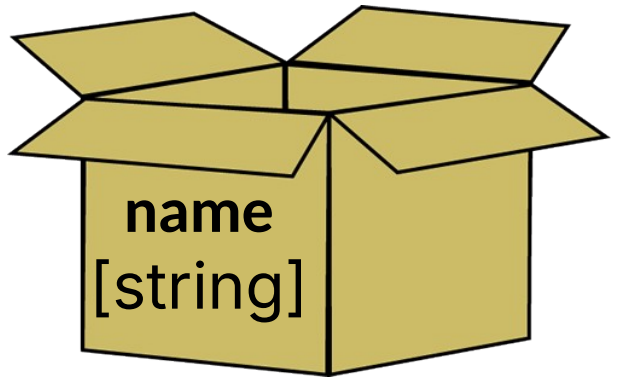
🔑 Key Point
Interactive programs follow a simple flow:
Input 👉 Process 👉 Output

Capturing Data with Console.ReadLine()

- Use **Conosle.ReadLine()** to pause the program until the user types text and presses Enter.

```
string name;
name = Console.ReadLine();
```

The Program waits for user input



🔑 Remember
Console.ReadLine() always returns data as a ‘string’, even if the user types in a number

Check Yourself (before you wreck yourself!)

To store a person's name, you first declare a _____string_____ variable, then use _____Console.ReadLine()_____ to capture their typing.

If you wanted to store their age, you would declare a variable called age with a data type of _____int_____.

To output the value stored within a variable, you would use the _____Console.WriteLine()_____ method.

Word Bank

String Console.WriteLine() int Console.ReadLine()

Making Choices: The IF Statement

Logic is about choices.

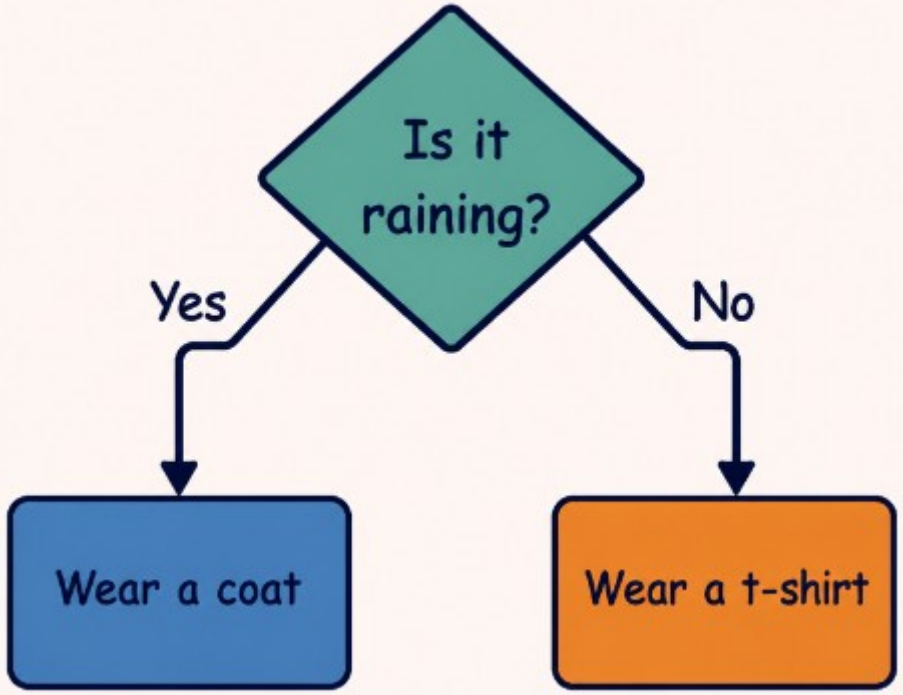
An IF statement tells the computer to run a block of code **only if** a specific condition is true. If it isn't true, we can use to provide a backup plan.

==

Checks if two values are equal

> and <

Checks if two values are greater or less than



Code Structure: Input + Log

⚠️ Watch Out!
Unlike in Maths, C# uses double equals sign (==) to compare things. A single equals sign (=) is only for assigning values.

The Code Pattern

```
string name = Console.ReadLine();

if(name == "Peter")
{
    Console.WriteLine("Hello Peter");
}
```

Within the brackets goes the condition

If the condition is true, then the code within the { and } runs.



The IF statement can also have an optional attachment called the ELSE statement.

It runs if the condition false

```
if (name == "Peter")
{
    Console.WriteLine("Hello Peter");
}
else
{
    Console.WriteLine("Hello Not Peter");
}
```

Operator Check

In C#, the code **if(age = 18)** is the correct way to check if someone is 18



TRUE





FALSE



Building a “Choose your Path” Story

Using what you've learned, create a mini-adventure. Ask the user: *'Do you go Left or Right?'*

1. Capture their choice in a variable.
2. Use an IF statement to print a different outcome for each path.

Challenge: The Age Checker

Can you solve this?

Write a program that asks for a user's age.

Use the operator to tell them if they are old enough to drive (17+ in the UK) or vote (18+).



🔑 Key Tip
To compare ages, you might need to convert the input using Convert.ToInt32()
But for now, try comparing string like if (age == “18”)

The Power of Repetition

Why do we need loops?

Imagine writing a program to print 'Hello World' 1,000 times. You could copy and paste 1,000 lines, but what if you needed to change the message later?

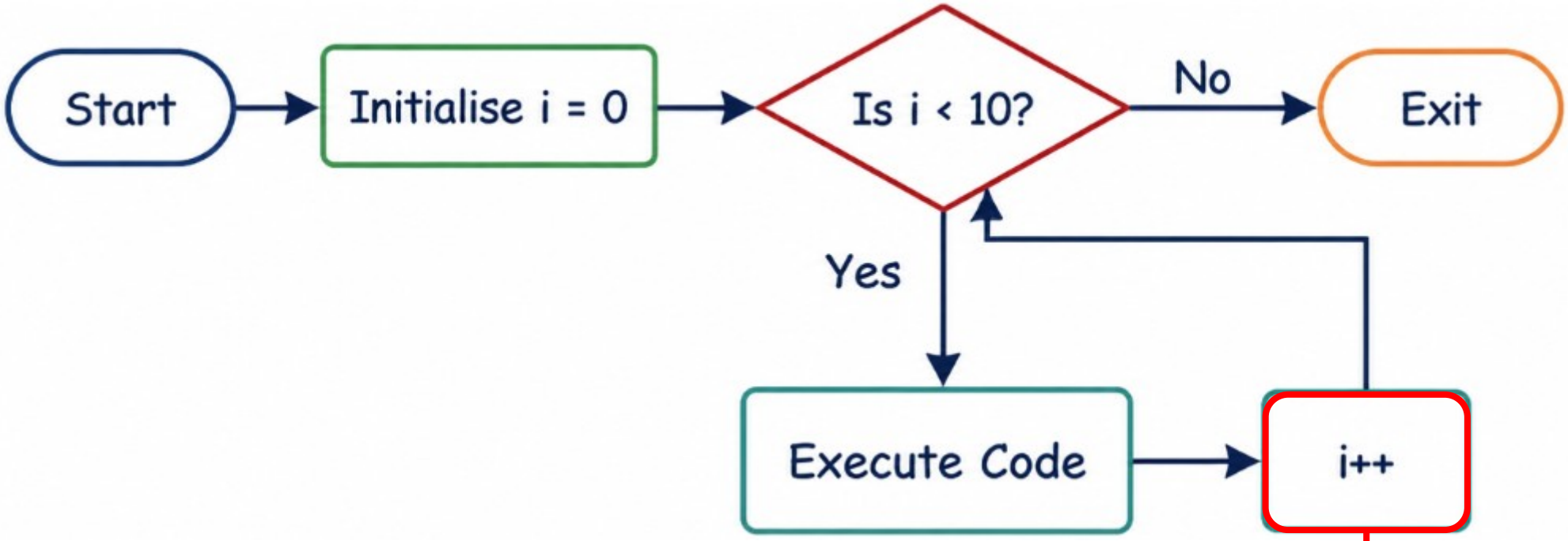
Loops allow us to write a block of code once and run it as many times as needed.



📖 Key Point
Loops make code shorter, easier to read, and much more maintainable.
This is the ‘DRY’ principle:
Don’t Repeat Yourself

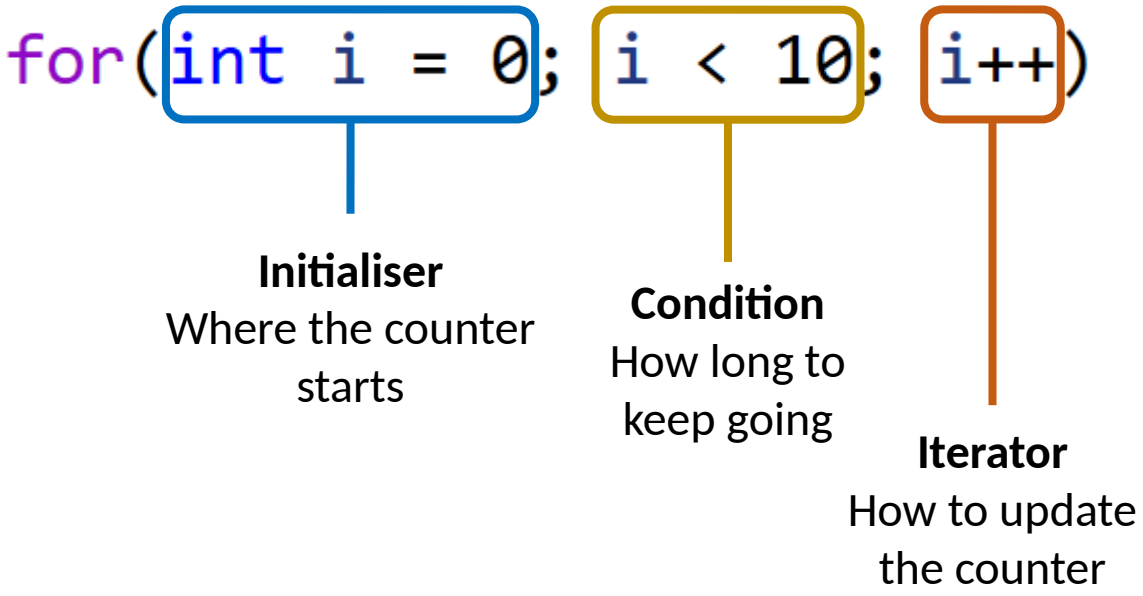
The FOR Loop Structure

A For loop is used when you know exactly how many times you want to repeat a task – this is called a **count-controlled loop**. It uses a counter to keep track of the repetitions.



This is a fancy looking piece of code. But all it does is increase (increment) i by 1.

Anatomy of a For Loop



In C#, the for loop has three parts:

1. **Initialiser**: Where the counter starts (e.g., int i = 0).
2. **Condition**: How long to keep going (e.g., i < 10).
3. **Iterator**: How to update the counter (e.g., i++).

📖 Example
Printing 1 to 5:

```
for(int i = 1; i < 5; i++)
{
    Console.WriteLine(i);
}
```

Check your Knowledge!

- Which part of the for loop defines when the loop should stop running?

The Initialiser	The Condition	The Iterator	The Curly Braces

- Which part of the for loop defines the counter?

The Initialiser	The Condition	The Iterator	The Curly Braces

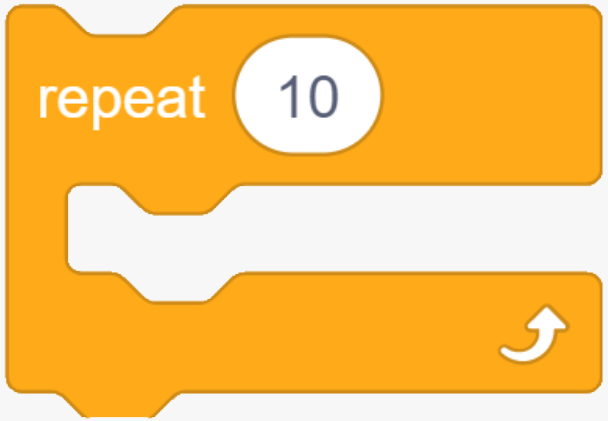
- Which part of the for loop increases the counter after each repetition?

The Initialiser	The Condition	The Iterator	The Curly Braces

Going around in circles? MORE LOOPS?

The FOR Loop

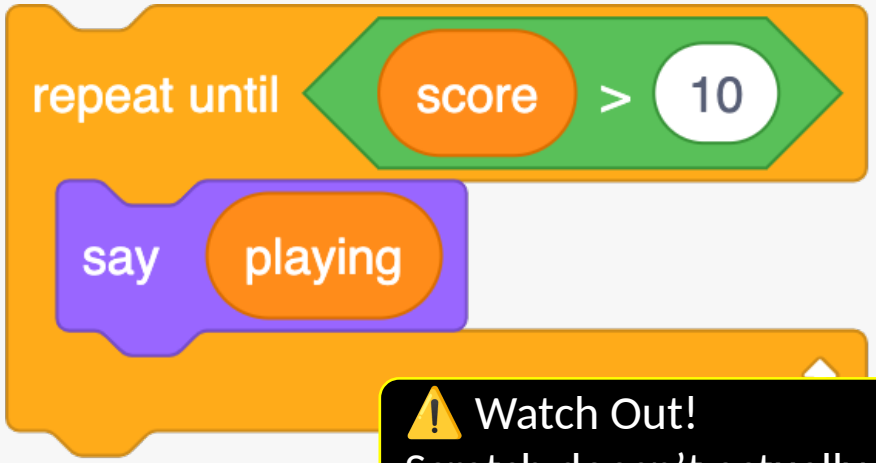
This is best for definite iteration, where we know how many times we want to repeat.





The WHILE Loop

This is best for indefinite iteration, where we don't know how many times it will run – like waiting for a user to enter the correct password





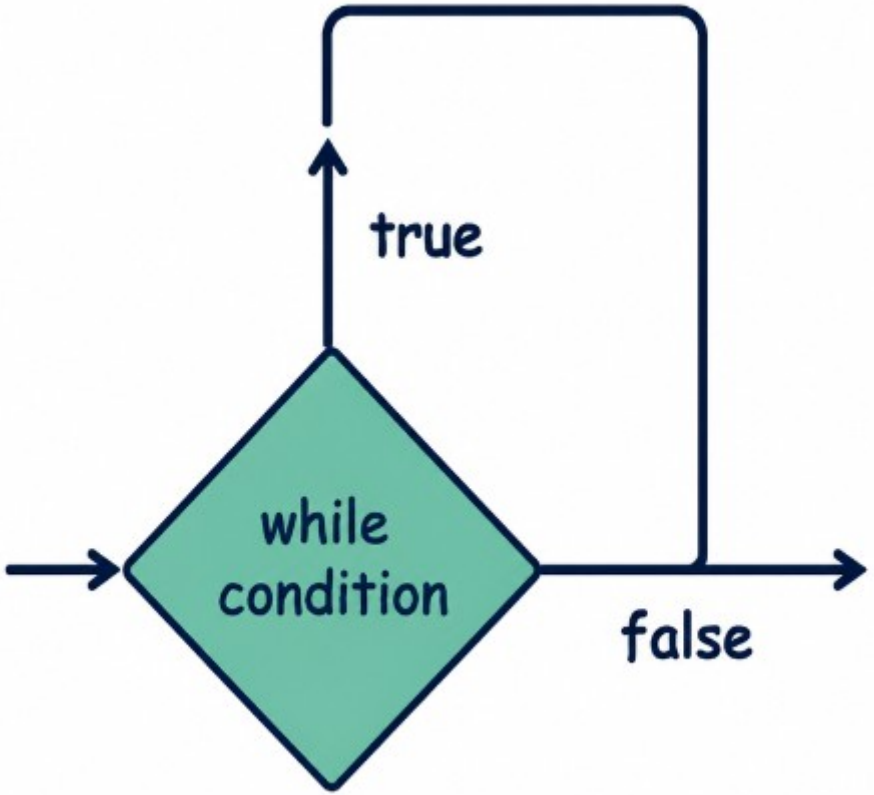
⚠ Watch Out!
Scratch doesn't actually have an explicit "WHILE" loop, so this repeat until loop actually does the opposite – when the condition becomes true.

Anatomy of a While Loop

The **while loop** checks a condition *before* executing the code block. If the condition is true, the code runs, then it checks again.

Logic Flow

- Check Condition
- Run Code (if true)
- Repeat until false




```
while( myName != "Mr B")
```

condition
Until this is false, the program will keep looping

The WHILE loop is fairly straightforward to write, as it looks a lot like the if statement we learnt previously. This time, however,

```
string myVar = Console.ReadLine();

while(myVar != "myPassword123")
{
    Console.WriteLine("Incorrect Password! Try again");
    myVar = Console.ReadLine();
}
```

📖 Key Point
!= means not equals to (≠) in C#
We don’t know how many times the user will enter in the password incorrectly, but this will repeat until the user enters in the correct password

Choosing the Right Tool

A key skill in programming is knowing *when* to use a **For Loop** (definite iteration) and when to use a **While Loop** (indefinite iteration).

Read the scenarios below. Decide which loop type is best and briefly explain why.

Scenario	Loop Type & Reason
Printing the 12 times table for a specific number.	For Loop (We know exactly how many times it needs to run - 12 times)
Waiting for a user to type in the correct password to log into a computer system.	While Loop (We don't know how many attempts the user will take)
A game where a player keeps guessing a secret number until they get it right.	While Loop (We don't know how many guesses it will take before the condition becomes true)

Task: Times Table Generator

Write a C# program using a **for loop** that asks the user for a number and then prints the multiplication table for that number from 1 to 12.

Challenge:

Use a **while loop** to keep asking for a new number until the user types 'exit'.

X	1	2	3	4	5	6	7	8	9	10	11	12
1	1	2	3	4	5	6	7	8	9	10	11	12
2	2	4	6	8	10	12	14	16	18	20	22	24
3	3	6	9	12	15	18	21	24	27	30	33	36
4	4	8	12	16	20	24	28	32	36	40	44	48
5	5	10	15	20	25	30	35	40	45	50	55	60
6	6	12	18	24	30	36	42	48	54	60	66	72
7	7	14	21	28	35	42	49	56	63	70	77	84
8	8	16	24	32	40	48	56	64	72	80	88	96
9	9	18	27	36	45	54	63	72	81	90	99	108
10	10	20	30	40	50	60	70	80	90	100	110	120
11	11	22	33	44	55	66	77	88	99	110	121	132
12	12	24	36	48	60	72	84	96	108	120	132	144

Match the Data!

Initialiser	A loop that never exists because the condition is always true.
Iteration	A Boolean expression that determines if a loop continues.
Condition	The starting value for a loop counter variable.
Infinite Loop	A single pass through a group of instructions.

Explain it to a 10 Year Old

One of the biggest dangers of a While loop is creating an **Infinite Loop**.

Imagine you have to explain what an Infinite Loop is to a 10-year-old. How would you describe it? Mention what causes it and what happens to the computer program as a result.

An infinite loop is like getting stuck on a roundabout and never being able to take the exit.

It happens when the condition to stop the loop never becomes false, so the computer keeps running the exact same code forever.

Eventually, it can cause the program to crash or freeze.

Challenge: The Guessing Game

How would you structure a loop for a game where the user has to guess a secret number between 1 and 100? Would you use a 'for' loop or a 'while' loop? Why?

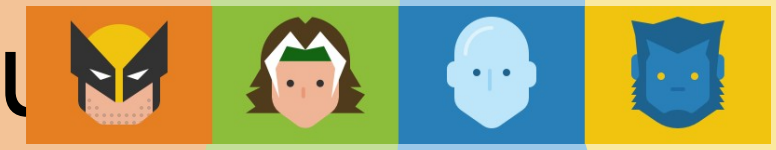
You might have said...

A while loop is best as we do not know the number of player attempts.

Update a boolean (e.g. isCorrect) inside the loop to end it upon success.

Use a counter inside the loop to track and display the total attempts.

The Core Fou

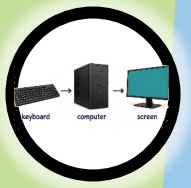


Imagine you are designing the next viral game or a secure bank login system.

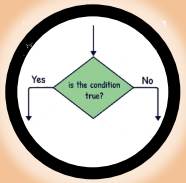
At their core, these complex systems rely on just four fundamental building blocks.



Variables
Storing data in memory locations like int score = 10;



I/O
Interacting with users via Console.ReadLine() and Console.WriteLine();



Selection
Using statements to make decisions based on conditions



Iteration
Using FOR or WHILE loops to repeat code efficiently.

🔑 Key Point
Programming is simply the art of giving a computer a set of instructions to solve a problem.

Match the Data!

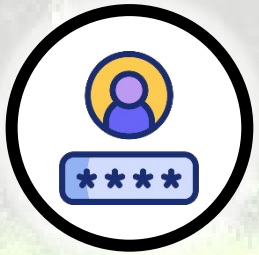
Variable	A data type that can only be true or false.
Selection	A named container for data that can change.
Boolean	Choosing which code to run based on a condition (IF).
Iteration	Repeating a section of code (looping).

Select Your Project!

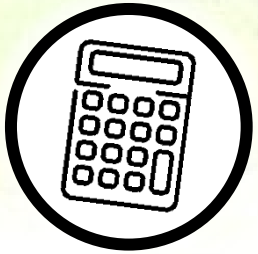
Choose one path to demonstrate your C# skills using the core concepts like variables and loops.



Greeting Bot
Ask for a name and age, then give a personalised welcome message.



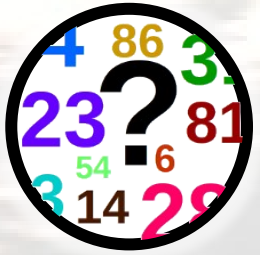
Login System
Compare a user's input against a stored password and grant or deny access.



Simple Calculator
Take two numbers and perform +,-,*, on them together using basic arithmetic



Fitness Tracker
Input daily steps and use a loop to calculate total steps over a week.



Number Guessing
Check if a user’s input matches a secret number using an if-statement.

STEP 1: The Planning Phas

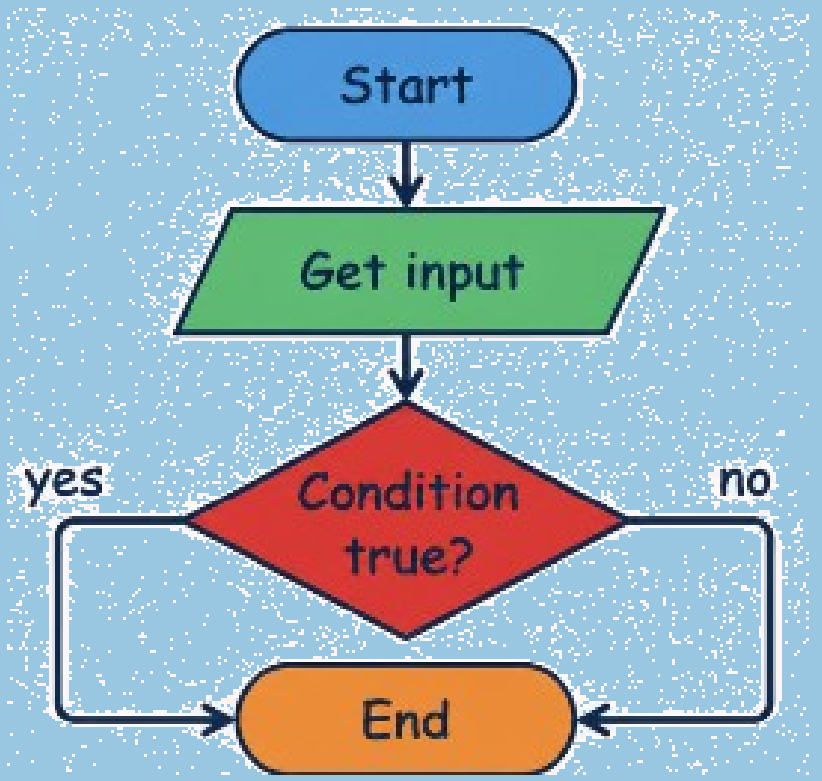
⚠️ Watch Out!
Starting code without a plan often leads to ‘spaghetti code’ that is hard to fix later.



Flowcharting

Don't touch the keyboard yet! Sketch your program flow.

- Where does the loop start?
- What condition triggers the IF statement?
- What variables do you need?



STEP 2: Building Iteratively

Code, Test, Repeat

1. **Setup**: Declare your variables first.
2. **Input**: Get the basic interaction working.
3. **Logic**: Add your IF statements.
4. **Loop**: Wrap your code in a loop last to avoid infinite loop errors during testing.

Clean Code Matters

Developers spend more time reading code than writing it. Your work is graded on readability.

Indentation

Keep code nested inside curly braces

Naming

Use meaningful names like number1 instead of xfgdfgdfgds or var1

Comments

Use // to explain complex logic

Review!



Swap seats with a partner. Try to 'break' their program by entering unexpected inputs (e.g., typing letters when it asks for numbers).

Give Feedback:

1. Does the loop end correctly?
2. Are the IF statements logical?
3. Is the code easy to read?

Final Review

Which of the four core skills (Variables, I/O, Selection, Iteration) did you find most challenging to implement in your project, and how did you overcome it?